

 COURSE

ADVANCED DATA SCIENCE LAB

CODE

MCA33PE21

ASSIGNMENT

06

Advanced Data Science Lab [MCA33PE21]

Practical Assignment - 06

K-Means, K-Nearest Neighbors, Naive Bayes and Hyperparameter Tuning

Field	Details
Academic Year	2026 – 2027
Class / Division	SYMCA (Semester-I)
PRN	125M1H026
Student Name	Shantanu Suryawanshi
Submission Date	06-10-2026

1. Implement and Evaluate Various Machine Learning Algorithms

This section implements and evaluates **K-Means Clustering**, **K-Nearest Neighbors (KNN)**, and **Naive Bayes** using the datasets provided for the practical.

Common preprocessing steps

- Load the dataset and inspect the first few rows.
- Check and handle missing values where necessary.
- Convert categorical target/features into a suitable form when required.
- Apply feature scaling for **K-Means** and **KNN**.
- Use an **80:20 train/test split** for supervised learning tasks.
- Evaluate classification models using appropriate metrics, confusion matrices, and classification reports.

Import Required Libraries

Only the libraries required for the given tasks are imported. The code is kept simple and divided into small cells so that each step can be understood and executed independently.

```
In [ ]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import MultinomialNB
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.pipeline import Pipeline
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    classification_report,
    ConfusionMatrixDisplay
)

RANDOM_STATE = 42
```

1(a). K-Means Clustering: Customer Segmentation

Problem Statement

Implement a K-Means clustering algorithm to segment customers based on their **annual income** and **spending score** using the **Mall Customer Segmentation dataset**.

Tasks

1. Select **Annual Income (k\$)** and **Spending Score (1-100)** as the clustering features.
2. Use the **Elbow Method** to determine the optimal number of clusters.
3. Train the K-Means model using the selected value of `k`.
4. Visualize the clusters using a scatter plot with a different color for each cluster.
5. Report the inertia score of the final model.

K-Means is an unsupervised learning algorithm, so a train/test split is not required.

Load and Inspect the Mall Customers Dataset

```
In [ ]: mall_customers = pd.read_csv("./data/Mall_Customers.csv")

print("Dataset shape:", mall_customers.shape)
mall_customers.head()
```

```
In [ ]: print("Missing values:")
        print(mall_customers.isnull().sum())

        print("\nData types:")
        print(mall_customers.dtypes)
```

Select Features and Scale the Data

Only the two features specified in the question are used. `CustomerID` is an identifier and `Gender` and `Age` are not required for this clustering task.

Feature scaling is applied because K-Means uses distances, and the selected numerical features should be on a comparable scale.

```
In [ ]: customer_features = mall_customers[
        ["Annual Income (k$)", "Spending Score (1-100)"]
        ]

        scaler = StandardScaler()
        scaled_customer_features = scaler.fit_transform(customer_features)

        print("Selected features:")
        print(customer_features.head())
```

Elbow Method

The Elbow Method compares the K-Means **inertia** for different values of `k`. The point where the decrease in inertia starts becoming smaller is used as a practical choice for the number of clusters.

```
In [ ]: inertia_values = []

        for k in range(1, 11):
            model = KMeans(
                n_clusters=k,
                random_state=RANDOM_STATE,
                n_init=10
            )
            model.fit(scaled_customer_features)
            inertia_values.append(model.inertia_)

        plt.figure(figsize=(8, 5))
        plt.plot(range(1, 11), inertia_values, marker="o")
        plt.xlabel("Number of Clusters (k)")
        plt.ylabel("Inertia")
        plt.title("Elbow Method for K-Means")
        plt.xticks(range(1, 11))
        plt.grid(True)
        plt.show()
```

Select the Optimal Number of Clusters

From the elbow plot, **k = 5** is selected for the final model. This provides a practical balance between reducing inertia and avoiding unnecessary additional clusters.

```
In [ ]: number_of_clusters = 5

kmeans_model = KMeans(
    n_clusters=number_of_clusters,
    random_state=RANDOM_STATE,
    n_init=10
)

cluster_labels = kmeans_model.fit_predict(scaled_customer_features)
mall_customers["Cluster"] = cluster_labels

print("Number of clusters:", number_of_clusters)
print("Final inertia:", kmeans_model.inertia_)
```

Visualize Customer Clusters

The scatter plot uses annual income on the x-axis and spending score on the y-axis. Each cluster is displayed separately so that the customer segments can be visually compared.

```
In [ ]: plt.figure(figsize=(9, 6))

for cluster in range(number_of_clusters):
    cluster_data = customer_features[cluster_labels == cluster]
    plt.scatter(
        cluster_data["Annual Income (k$)"],
        cluster_data["Spending Score (1-100)"],
        label=f"Cluster {cluster + 1}"
    )

plt.xlabel("Annual Income (k$)")
plt.ylabel("Spending Score (1-100)")
plt.title("Customer Segmentation using K-Means")
plt.legend()
plt.grid(True)
plt.show()
```

K-Means Finding

The Elbow Method indicates that **5 clusters** are a suitable choice for this dataset. The final K-Means model is therefore trained with `k = 5`, and its inertia is reported above. The scatter plot shows the resulting customer segments based on annual income and spending score.

1(b). K-Nearest Neighbors (KNN): Iris

Classification

Problem Statement

Build a **K-Nearest Neighbors (KNN)** classifier to predict the species of an Iris flower — **setosa**, **versicolor**, or **virginica** — using its sepal and petal measurements.

Tasks

1. Implement a KNN classifier.
2. Train the model and make predictions on the test set.
3. Evaluate the model using **accuracy**, **precision**, **recall**, and **F1-score**.

Load and Inspect the Iris Dataset

```
In [ ]: iris_data = pd.read_csv("./data/Iris.csv")

print("Dataset shape:", iris_data.shape)
iris_data.head()
```

```
In [ ]: print("Missing values:")
print(iris_data.isnull().sum())

print("\nSpecies values:")
print(iris_data["Species"].unique())
```

Prepare the Data and Split into Training and Testing Sets

The `Id` column is only an identifier, so it is not used as a feature. The four sepal and petal measurements are used as input features.

The data is divided into **80% training** and **20% testing** data. `stratify` keeps the class proportions similar in both sets.

```
In [ ]: iris_features = iris_data[
    ["SepalLengthCm", "SepalWidthCm", "PetalLengthCm", "PetalWidthCm"]
]
iris_target = iris_data["Species"]

X_train, X_test, y_train, y_test = train_test_split(
    iris_features,
    iris_target,
    test_size=0.20,
    random_state=RANDOM_STATE,
    stratify=iris_target
)

print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))
```

Feature Scaling and KNN Model

KNN is distance-based, so the numerical features are standardized before classification. A small pipeline keeps scaling and classification together and prevents accidental differences between training and testing preprocessing.

```
In [ ]: knn_model = Pipeline([
        ("scaler", StandardScaler()),
        ("knn", KNeighborsClassifier(n_neighbors=5))
    ])

knn_model.fit(X_train, y_train)
knn_predictions = knn_model.predict(X_test)

print("KNN model trained successfully.")
```

Evaluate the KNN Model

```
In [ ]: knn_accuracy = accuracy_score(y_test, knn_predictions)
knn_precision = precision_score(
    y_test, knn_predictions, average="weighted"
)
knn_recall = recall_score(
    y_test, knn_predictions, average="weighted"
)
knn_f1 = f1_score(
    y_test, knn_predictions, average="weighted"
)

print("KNN Accuracy :", knn_accuracy)
print("KNN Precision:", knn_precision)
print("KNN Recall   :", knn_recall)
print("KNN F1-Score :", knn_f1)

print("\nClassification Report:")
print(classification_report(y_test, knn_predictions))
```

```
In [ ]: ConfusionMatrixDisplay.from_predictions(
    y_test,
    knn_predictions,
    xticks_rotation=45
)
plt.title("KNN Confusion Matrix")
plt.show()
```

KNN Finding

The KNN classifier is evaluated on unseen test data using accuracy, precision, recall, and F1-score. The confusion matrix shows the correctly and incorrectly classified Iris species.

1(c). Naive Bayes Classification: SMS Spam Detection

Problem Statement

Develop a **Naive Bayes classifier** to predict whether an SMS message is **ham (not spam)** or **spam** using the SMS Spam Collection dataset.

Tasks

1. Convert the text into numerical features using a technique such as **CountVectorizer** or **TfidfVectorizer**.
2. Implement a **MultinomialNB** or **GaussianNB** classifier.
3. Train and evaluate the model.
4. Pay close attention to precision and recall for the minority **spam** class because the dataset is imbalanced.
5. Improve the scores using an appropriate technique for the imbalanced dataset.

Load and Inspect the SMS Spam Dataset

The supplied CSV contains the message label in **v1** and the SMS text in **v2**. The extra **Unnamed** columns do not contain useful information for this task and are ignored.

```
In [ ]: spam_data = pd.read_csv("./data/spam.csv", encoding="latin-1")

print("Dataset shape:", spam_data.shape)
spam_data[["v1", "v2"]].head()
```

```
In [ ]: print("Missing values:")
print(spam_data[["v1", "v2"]].isnull().sum())

print("\nClass distribution:")
print(spam_data["v1"].value_counts())
```

Prepare Text Data and Train/Test Split

The SMS text is converted into numerical features using **TF-IDF**. A stratified 80:20 split is used so that both training and testing data retain the original ham/spam class proportions.

```
In [ ]: message_text = spam_data["v2"].fillna("")
message_label = spam_data["v1"]

X_train_text, X_test_text, y_train_text, y_test_text = train_test_split(
    message_text,
    message_label,
    test_size=0.20,
    random_state=RANDOM_STATE,
    stratify=message_label
)

print("Training messages:", len(X_train_text))
print("Testing messages:", len(X_test_text))
```

Baseline Multinomial Naive Bayes Model

`TfidfVectorizer` converts the SMS text into numerical TF-IDF features, and `MultinomialNB` performs the classification.

```
In [ ]: spam_model = Pipeline([
    ("tfidf", TfidfVectorizer()),
    ("naive_bayes", MultinomialNB())
])

spam_model.fit(X_train_text, y_train_text)
spam_predictions = spam_model.predict(X_test_text)

print("Baseline Naive Bayes model trained successfully.")
```

```
In [ ]: print("Baseline Accuracy :", accuracy_score(y_test_text, spam_predictions))
print("Baseline Precision:", precision_score(
    y_test_text, spam_predictions, pos_label="spam"
))
print("Baseline Recall   :", recall_score(
    y_test_text, spam_predictions, pos_label="spam"
))
print("Baseline F1-Score :", f1_score(
    y_test_text, spam_predictions, pos_label="spam"
))

print("\nBaseline Classification Report:")
print(classification_report(y_test_text, spam_predictions))
```

```
In [ ]: ConfusionMatrixDisplay.from_predictions(
    y_test_text,
    spam_predictions,
    display_labels=["ham", "spam"]
)
plt.title("Baseline Naive Bayes Confusion Matrix")
plt.show()
```

Improve the Model for the Imbalanced Dataset

The dataset contains many more `ham` messages than `spam` messages. To give the minority class greater importance, the Multinomial Naive Bayes model is trained with **equal class priors** using `class_prior=[0.5, 0.5]`.

This is a simple adjustment that does not require an additional library. The spam precision and recall are compared with the baseline model.

```
In [ ]: balanced_spam_model = Pipeline([
    ("tfidf", TfidfVectorizer()),
    ("naive_bayes", MultinomialNB(class_prior=[0.5, 0.5]))
])

balanced_spam_model.fit(X_train_text, y_train_text)
balanced_spam_predictions = balanced_spam_model.predict(X_test_text)

print("Improved Naive Bayes model trained successfully.")
```

```
In [ ]: baseline_spam_recall = recall_score(
    y_test_text, spam_predictions, pos_label="spam"
)
improved_spam_recall = recall_score(
    y_test_text, balanced_spam_predictions, pos_label="spam"
)

baseline_spam_f1 = f1_score(
    y_test_text, spam_predictions, pos_label="spam"
)
improved_spam_f1 = f1_score(
    y_test_text, balanced_spam_predictions, pos_label="spam"
)

print("Improved Accuracy :", accuracy_score(
    y_test_text, balanced_spam_predictions
))
print("Improved Precision:", precision_score(
    y_test_text, balanced_spam_predictions, pos_label="spam"
))
print("Improved Recall   :", improved_spam_recall)
print("Improved F1-Score :", improved_spam_f1)

print("\nImproved Classification Report:")
print(classification_report(y_test_text, balanced_spam_predictions))
```

```
In [ ]: ConfusionMatrixDisplay.from_predictions(
        y_test_text,
        balanced_spam_predictions,
        display_labels=["ham", "spam"]
    )
plt.title("Improved Naive Bayes Confusion Matrix")
plt.show()
```

```
In [ ]: spam_comparison = pd.DataFrame({
    "Model": ["Baseline Naive Bayes", "Improved Naive Bayes"],
    "Accuracy": [
        accuracy_score(y_test_text, spam_predictions),
        accuracy_score(y_test_text, balanced_spam_predictions)
    ],
    "Spam Recall": [
        baseline_spam_recall,
        improved_spam_recall
    ],
    "Spam F1-Score": [
        baseline_spam_f1,
        improved_spam_f1
    ]
})

print(spam_comparison.to_string(index=False))

spam_comparison.plot(
    x="Model",
    y=["Accuracy", "Spam Recall", "Spam F1-Score"],
    kind="bar",
    figsize=(9, 5)
)
plt.title("Baseline vs Improved Naive Bayes")
plt.ylabel("Score")
plt.ylim(0, 1.05)
plt.xticks(rotation=0)
plt.grid(axis="y")
plt.show()
```

Naive Bayes Finding

The baseline model performs well overall, but its spam recall is lower because the dataset is imbalanced. Using equal class priors increases the importance of the minority `spam` class and improves its recall and F1-score on this dataset. This demonstrates why class-sensitive evaluation is important instead of relying only on overall accuracy.

2. Hyperparameter Tuning with GridSearchCV

2(a). Improve the K-Nearest Neighbors (KNN) Model Using GridSearchCV

The KNN model from **Question 1(b)** is tuned using `GridSearchCV`.

Tasks

1. Use the same Iris dataset and KNN model.
2. Define a parameter grid containing at least two hyperparameters.
3. Tune `n_neighbors` and `weights`.
4. Use **5-fold cross-validation**.
5. Fit `GridSearchCV` to the training data.
6. Print the best parameters and best cross-validation score.
7. Compare the tuned model's test accuracy with the default KNN model.
8. Summarize how hyperparameter tuning affected performance.

Define the Parameter Grid

Two KNN hyperparameters are tested:

- `n_neighbors` : number of nearby samples considered for classification.
- `weights` : whether all neighbors have equal importance (`uniform`) or closer neighbors receive greater importance (`distance`).

```
In [ ]: parameter_grid = {  
        "knn__n_neighbors": [3, 5, 7, 9, 11],  
        "knn__weights": ["uniform", "distance"]  
    }  
  
    print("Parameter grid:")  
    print(parameter_grid)
```

Apply 5-Fold GridSearchCV

The same scaling step and KNN classifier are used inside a pipeline. Five-fold cross-validation is used to compare the parameter combinations on the training data.

```
In [ ]: grid_search = GridSearchCV(
    estimator=Pipeline([
        ("scaler", StandardScaler()),
        ("knn", KNeighborsClassifier())
    ]),
    param_grid=parameter_grid,
    cv=5,
    scoring="accuracy"
)

grid_search.fit(X_train, y_train)

print("Best parameters:", grid_search.best_params_)
print("Best cross-validation accuracy:", grid_search.best_score_)
```

Evaluate and Compare Default KNN with Tuned KNN

```
In [ ]: tuned_predictions = grid_search.predict(X_test)

default_accuracy = accuracy_score(y_test, knn_predictions)
tuned_accuracy = accuracy_score(y_test, tuned_predictions)

print("Default KNN test accuracy:", default_accuracy)
print("Tuned KNN test accuracy :", tuned_accuracy)

if tuned_accuracy > default_accuracy:
    print("The tuned KNN model improved the test accuracy.")
elif tuned_accuracy < default_accuracy:
    print("The tuned KNN model did not improve the test accuracy on this
test split.")
else:
    print("Both models achieved the same test accuracy on this test split.")
```

```
In [ ]: accuracy_comparison = pd.DataFrame({
    "Model": ["Default KNN", "Tuned KNN"],
    "Test Accuracy": [default_accuracy, tuned_accuracy]
})

print(accuracy_comparison.to_string(index=False))

plt.figure(figsize=(7, 5))
plt.bar(accuracy_comparison["Model"], accuracy_comparison["Test Accuracy"])
plt.ylabel("Accuracy")
plt.title("Default KNN vs Tuned KNN")
plt.ylim(0, 1.05)
plt.grid(axis="y")
plt.show()
```

```
In [ ]: print("Tuned KNN Classification Report:")
        print(classification_report(y_test, tuned_predictions))

        ConfusionMatrixDisplay.from_predictions(
            y_test,
            tuned_predictions,
            xticks_rotation=45
        )
        plt.title("Tuned KNN Confusion Matrix")
        plt.show()
```

Overall Findings

- **K-Means:** The Elbow Method was used to select a practical value of `k`, and the final clusters were visualized using annual income and spending score.
- **KNN:** The Iris dataset was classified using scaled numerical features and evaluated with accuracy, precision, recall, F1-score, and a confusion matrix.
- **Naive Bayes:** TF-IDF was used to convert SMS text into numerical features. Because the dataset is imbalanced, spam precision, recall, and F1-score were considered in addition to accuracy. Equal class priors improved minority-class performance on this dataset.
- **GridSearchCV:** KNN hyperparameters were evaluated using 5-fold cross-validation. The best parameters and cross-validation score were reported, and the tuned model was compared with the default KNN model on the test set.

Declaration

I hereby declare that the work submitted in this practical assignment is my own and has not been copied from any other source.

Student Name	Shantanu Suryawanshi
PRN	125M1H026
Date	06-10-2026

Prof. Prakash Ukhalkar

Course Teacher - Advanced Data Science Lab [MCA33PE21]

MCA33PE21 Advanced Data Science Lab

Practical Assignment 06